oneAPI DPC++ Library (oneDPL)

oneDPL development team

UXL
Unified Acceleration Foundation

Introduction to oneDPL

High productivity C++ library to reduce programming efforts for parallel & heterogeneous applications.

Implements C++ standard API abstractions for data parallel devices.

SYCL, OpenMP, and oneTBB backends enable parallel execution on CPUs, GPUs and other accelerators.

Specification (latest, tentatively v1.5): https://uxlfoundation.github.io/oneAPI-spec/spec/elements/oneDPL/source/index.html

Source code: https://github.com/uxlfoundation/oneDPL

Documentation: https://uxlfoundation.github.io/oneDPL

Components of oneDPL

Parallel & Heterogeneous Execution API

- Parallel Algorithms
- Parallel Range Algorithms
- Execution Policies
- Auxiliary Functions
- Dynamic Selection
- Kernel Templates

Additional API

Special Iterators

- zip
- transform
- counting
- …

Tested C++ API for Kernels

- <cmath> <complex>
- <type_traits> <functional>
- …

Random Number Generation

- Engines
- Distributions

Projects using oneDPL

- NAMD
- AMBER
- HYPRE
- Ginkgo
- ArborX
- Nabla
- Kokkos
- dissolve
- congenial-enigma
- Schron
- AMREX
- alpaka

Basic oneDPL usage

- Requires oneTBB, OpenMP, or SYCL for parallelism
- C++17 is the minimal supported version of the C++ standard
  - Some APIs might require a higher version
- Header names start with `oneapi/dpl`:
  `#include <oneapi/dpl/algorithm>`
- All functionality is provided in `namespace oneapi::dpl`
  - short alias: `namespace dpl = oneapi::dpl;`

oneDPL API Overview

UXL
Unified Acceleration Foundation

Parallel algorithms in C++

… with C++17 execution policies

```
#include <algorithm>
#include <vector>


int main()
{
    std::vector<int> v(100000);
    /* The fill algorithm w/o execution policy  */
    std::fill(v.begin(), v.end(), 42);
}
```

```
#include <algorithm>
#include <vector>
#include <execution>

int main()
{
    std::vector<int> v(100000);
    /* The fill algorithm with parallel execution policy */
    std::fill(std::execution::par, v.begin(), v.end(), 42);
}
```

UXL
Unified Acceleration Foundation

Parallel algorithms in oneDPL

...with device execution policy

```
#include <algorithm>
#include <execution>
#include <vector>

int main()
{
    std::vector<int> v(100000);

    /* The fill algorithm with parallel execution policy */
    std::fill(std::execution::par, v.begin(), v.end(), 42);
}
// icpx std_example.cpp
```

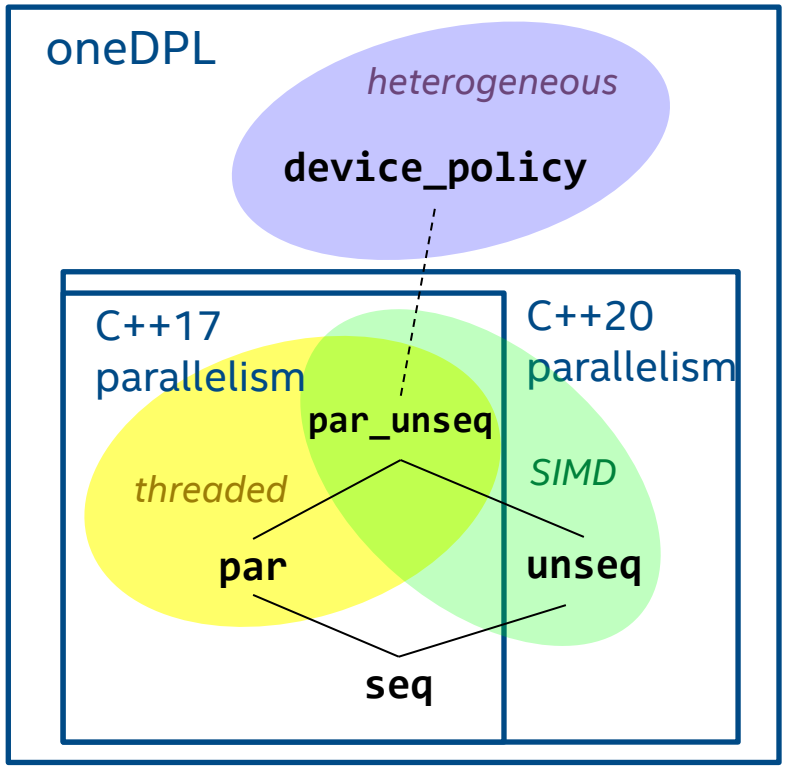
```
#include <oneapi/dpl/algorithm>
#include <oneapi/dpl/execution>
#include <vector>

int main()
{
    std::vector<int> v(100000);

    /* The fill algorithm with device execution policy*/
    oneapi::dpl::fill(oneapi::dpl::execution::dpcpp_default,
                      v.begin(), v.end(), 42);
}
// icpx -fsycl dpl_example.cpp
```

All 80+ functions with execution policies in `<algorithm>`, `<numeric>`, and `<memory>`.

UXL
Unified Acceleration Foundation

oneDPL Execution Policies

- Standard-aligned policies for CPU execution: `oneapi::dpl::execution::{seq|unseq|par|par_unseq}`
- `oneapi::dpl::execution::device_policy` class template
  - Encapsulates a SYCL* queue to specify where algorithms should run
- `oneapi::dpl::execution::dpcpp_default`
  - A predefined `device_policy` instance using the default SYCL queue



Parallel algorithms in oneDPL

...with device execution policy & SYCL unified shared memory

```
#include <oneapi/dpl/algorithm>
#include <oneapi/dpl/execution>
#include <vector>

int main()
{
    std::vector<int> v(100000);

    /* The fill algorithm with device execution policy*/
    oneapi::dpl::fill(oneapi::dpl::execution::dpcpp_default,
                      v.begin(), v.end(), 42);
}
// icpx -fsycl dpl_example.cpp
```

```
#include <oneapi/dpl/algorithm>
#include <oneapi/dpl/execution>
#include <vector>
#include <sycl/sycl.hpp> // just to be explicit

int main()
{
    sycl::queue q{sycl::gpu_selector_v};
    sycl::usm_allocator<int, sycl::usm::shared> alloc{q};
    std::vector v(100000, alloc); // Data are accessible on GPU

    /* The fill algorithm with execution policy for GPU */
    oneapi::dpl::fill(oneapi::dpl::execution::device_policy{q},
                      v.data(), v.data() + v.size(), 42);
}
// icpx -fsycl dpl_usm_example.cpp
```

Note: an algorithm call blocks until the work is complete on the device.

UXL
Unified Acceleration Foundation

C++ standard parallelism on SYCL devices

Use `-fsycl-pstl-offload` option of Intel® DPC++ Compiler:

- Redirects C++ standard algorithms with `par_unseq` to a device (GPU/CPU/default)
- Redirects heap allocated memory to USM accessible at that device
- Without changing the code! (limitations apply though)

- Uses oneDPL parallel algorithms underneath

UXL
Unified Acceleration Foundation

Parallel algorithms with PSTL offload

… with Intel® DPC++ Compiler and (implicitly) oneDPL

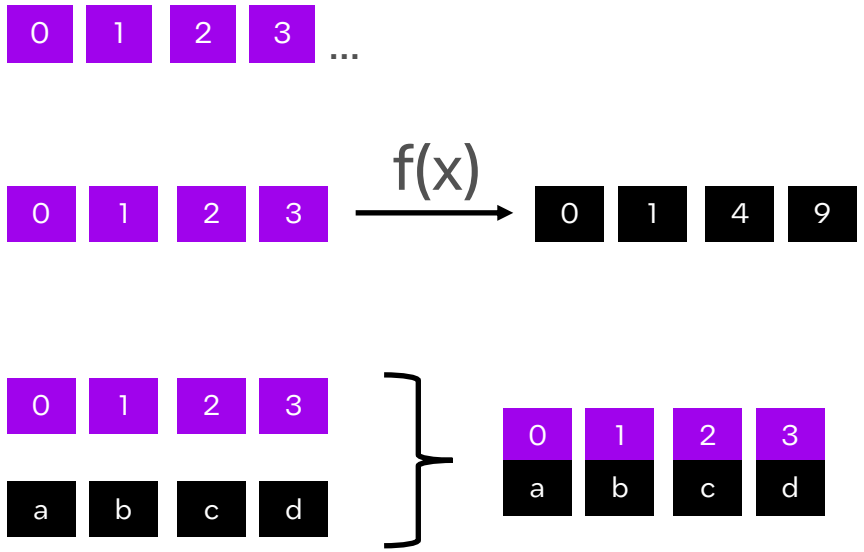
```
#include <algorithm>
#include <execution>
#include <vector>

int main()
{
    std::vector<int> v(100000);

    /* The fill algorithm with parallel execution policy */
    std::fill(std::execution::par, v.begin(), v.end(), 42);
}
// icpx std_example.cpp
```

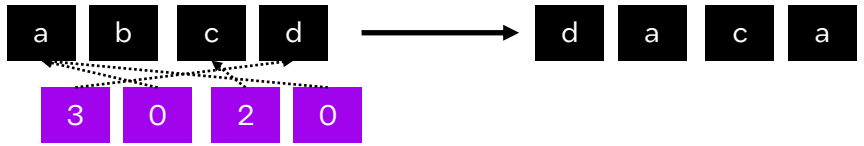
```
#include <algorithm>
#include <execution>
#include <vector>

int main()
{
    std::vector<int> v(100000);

    /* The fill algorithm with device execution policy*/
    std::fill(std::execution::par_unseq, v.begin(), v.end(), 42);
}
// icpx -fsycl -fsycl-pstl-offload=gpu pstl_offload_example.cpp
// Supported values: [cpu|gpu]
```

UXL
Unified Acceleration Foundation

Iterators for broader applicability

- `counting_iterator`
  - Generates a sequence of indexes “on the fly”
- `transform_iterator`
  - Applies a function via another iterator
- `zip_iterator`
  - Combines multiple iterators into one
- `permutation_iterator`
  - Iterates in the order defined by another sequence or a function





Other useful functionality

- Additional oneDPL algorithms
  - Reduce and scan by segments
  - Binary search in batches
  - Sort by key
  - transform_if
  - Histogram
- Support for SYCL buffers
  - `oneapi::dpl::begin/end` to pass buffers to the algorithms
- Random number generation
  - Standard-compliant APIs
  - Suitable for SYCL kernels
  - Can operate with `sycl::vec`

Parallel Range Algorithms

Ranges, huh?

- **Range** is an abstraction over an iteration sequence
  - Essentially, a pair of an iterator and a *sentinel*: `[first, last)`
- **View** is a range that can be "copied" in constant time
  - No data copying is involved
  - *Range adaptors* modify data "on the fly" (e.g., `std::views::transform`)
  - *Factories* generate data "on the fly" (e.g., `std::views::iota`)
- Ranges and views are easy to compose into "lazy" data pipelines
- **Range algorithms** are C++20/23 algorithms that operate on ranges
  - Parameters are constrained by C++ concepts
  - Support *projections* of the range elements

A conceptual example

Let's apply a given function to a sequence and then find the last element, which value satisfies a given predicate

```
std::transform(std::begin(data), std::end(data), std::begin(modified), func);
std::reverse(std::begin(modified), std::end(modified));
auto res = std::find_if(std::begin(modified), std::end(modified), pred);
```

With the ranges API, this can be expressed as

```
auto modified = data | std::views::transform(func); // no extra memory is required!
auto res = std::ranges::find_if(modified | std::views::reverse, pred);

// or even std::ranges::find_last_if(modified, pred);
```

The example: let’s go parallel

What if there is a lot of data to crunch?

```
std::transform(policy, std::begin(data), std::end(data), std::begin(modified), func);
std::reverse(policy, std::begin(modified), std::end(modified));
auto res = std::find_if(policy, std::begin(modified), std::end(modified), pred);
```

- An execution policy added to the serial STL variant

But:

- Extra memory for modified data
- Three parallel algorithm invocations, each adding its own overhead
- “Early exit” only possible for the third algorithm call

Special iterators can be useful

```
auto res = std::find_if(policy,
    std::make_reverse_iterator(dpl::make_transform_iterator(std::end(data), func)),
    std::make_reverse_iterator(dpl::make_transform_iterator(std::begin(data), func)),
    pred);
```

- No extra memory
- One algorithm invocation, less overhead for parallelism
- The unnecessary work might be skipped by early exit

But:

- Extra verbosity and complexity
- E.g., `end(data)` is a `reverse_iterator` begin

Using ranges with existing algorithms

But we could get iterators from a range, right?

```
auto mod_reversed = data | std::views::transform(func) | std::views::reverse;
auto res = std::find_if(policy, std::ranges::begin(mod_reversed),
                        std::ranges::end(mod_reversed), pred);
```

- Probably the best that we can do with Parallel STL

But:

- Still some extra verbosity
- For our example, “embedded” reversing makes the pipeline less reusable

Finally, with parallel range algorithms

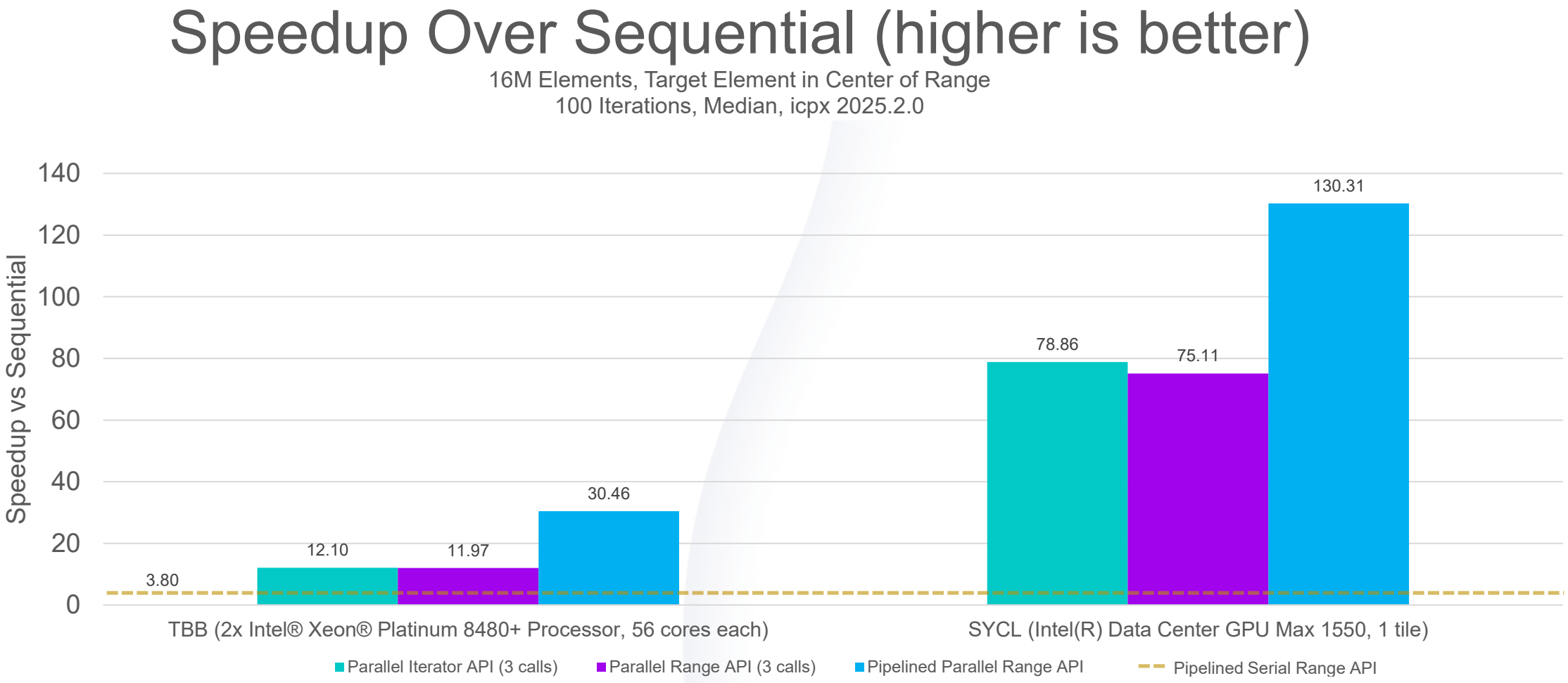
```
auto modified = data | std::views::transform(func);
auto res = dpl::ranges::find_if(policy, modified | std::views::reverse, pred);

// or even dpl::ranges::find_last_if(policy, modified, pred);
```

- Essentially, an execution policy added to the serial ranges variant
- No extra memory
- One algorithm invocation, less overhead for parallelism
- The unnecessary work might be skipped by early exit
- Concise and expressive!

Heterogeneous with oneDPL

```
sycl::queue q{sycl::gpu_selector_v};
// Using USM allocator for memory access on the host and the device
using alloc_type = sycl::usm_allocator<std::size_t, sycl::usm::alloc::shared>;
alloc_type alloc{q};

// Preparing data
std::vector<std::size_t, alloc_type> v{size, alloc};
std::ranges::subrange data{v};

auto modified = data | std::views::transform(func);
auto res = dpl::ranges::find_last_if(dpl::make_device_policy(q), modified, pred);
```

Performance



UXL
Unified Acceleration Foundation

Parallel range algorithms: Summary

- Combine the composability and convenience of C++ ranges API with parallelism
- Opportunity to reduce overheads by “fusing” algorithm invocations
- Better expressiveness and productivity for parallel code

- 78 new algorithms added to the oneDPL specification (v1.5)
- 2/3 already implemented
  - Supporting standard range & view types
- Our standardization proposal for parallel range algorithms is <u>accepted for C++26</u>

Experimental Functionality

oneDPL Dynamic Selection API

- Sometimes developers must manually choose which device to run some work on
  - May not have enough experience to select among the available devices
  - May not know the current load on a particular device or know the details of performance while choosing between devices

- The Dynamic Selection API is an experimental feature in oneDPL that helps choosing a resource using a *selection policy*
  - To provide load balancing across equivalent devices, e.g. multiple GPUs
  - To select an appropriate choice given disparate devices, e.g. CPU or GPU

Syntactic example

```
#include <oneapi/dpl/dynamic_selection>
#include <sycl/sycl.hpp>
#include <iostream>

namespace ex = oneapi::dpl::experimental;

int main() {

  // (1) create a policy object
  ex::round_robin_policy p{ { sycl::queue{ sycl::cpu_selector_v },
                              sycl::queue{ sycl::gpu_selector_v } } };

  for (int i = 0; i < 6; ++i) {

    // (2) call one of the dynamic selection functions
    //     -- pass the policy to the API function
    //     -- provide a function to be called with a selected queue
    //        -- the user function must receive a sycl queue
    //        -- the user function must return a sycl event
    auto done =
      ex::submit(p,
                 // (3) use the selected queue in user function
                 [=](sycl::queue q) {
                   std::cout << "submit task to "
                             << ((q.get_device().is_gpu())
                                 ? "gpu\n" : "cpu\n");
                   return q.single_task([]() { /* work here */ });
                 });

    // (4) each submission can be waited on using the returned object
    ex::wait(done);
  }

  // (5) and/or all submissions can be waited on as a group
  ex::wait(p.get_submission_group());
}
```

```
submit task to cpu
submit task to gpu
submit task to cpu
submit task to gpu
submit task to cpu
submit task to gpu
```

oneDPL selection policies

Policy	Description	Execution Info Required
fixed_resource_policy	Always selects a specified resource. Good for debugging, or to hard-code choice for specific platform.	none
round_robin_policy	Rotates through set at each selection point. Can be used to balance load.	none
dynamic_load_policy	Selects least loaded device at selection time.	Uses task submission and task completion callbacks.
auto_tune_policy	Selects “best” device based on runtime profiling.	Uses task execution time. If queue profiling is enabled, uses event timings.

oneDPL Kernel Templates API

- oneDPL API is mostly defined by the C++ standard
  - High-level, generic, productivity oriented
- Sometimes users prefer performance, even at the cost of productivity & portability
  - Want more tuning control
  - Open to use HW specific implementation
- The experimental Kernel Templates API attempts to address that demand via:
  - Focusing strictly on SYCL as the execution model
  - Compile-time choice of algorithm-specific parameters
  - Compile-time data partitioning: workgroup size, iterations per work item

Syntactic example: Radix sort by key

```
#include <oneapi/dpl/experimental/kernel_templates>
namespace kt = oneapi::dpl::experimental::kt;

int main()
{
   std::size_t n = 6;
   sycl::queue q{sycl::gpu_selector_v};
   sycl::buffer<std::uint32_t> keys{sycl::range<1>(n)};
   sycl::buffer<char> values{sycl::range<1>(n)};

   { // initialize data on the host
      sycl::host_accessor k_acc{keys, sycl::write_only};
      k_acc[0] = 3, k_acc[1] = 2, k_acc[2] = 1, k_acc[3] = 5, k_acc[4] = 3, k_acc[5] = 3;

      sycl::host_accessor v_acc{values, sycl::write_only};
      v_acc[0] = 'r', v_acc[1] = 'o', v_acc[2] = 's', v_acc[3] = 'd', v_acc[4] = 't', v_acc[5] = 'e';
   }

   // (1) explicitly asynchronous, returns an event
   // (2) nested namespaces indicate targeted platforms
   // (3) algorithm-specific customization via template arguments
   sycl::event e = kt::gpu::esimd::radix_sort_by_key</*ascending*/true, /*radix*/8>(
                       q, keys, values,
                       // (4) customize data partitioning with a special function argument
                       kt::kernel_param</*data per work item*/1, /*workgroup size*/16>{});
   e.wait();

   // The buffers now contain:
   // keys:   1 2 3 3 3 5
   // values: s o r t e d
}
```

You can contribute, too!☺

Contributing to oneDPL

oneDPL has a code contribution process, discussions for exchange of ideas, and RFCs for specific proposals.

Possible contribution areas:

- Feedback on the open RFCs and discussions
- Testing on various hardware types
- New examples using oneDPL
- Improve test input size coverage
- OpenMP backend improvements
- New experimental algorithms (esp. kernel templates) and backends

More Materials and Examples

Buffer Wrappers

- `oneapi::dpl::begin` and `oneapi::dpl::end` are helper functions for passing SYCL buffers to oneDPL algorithms
- Applied to a SYCL buffer, these functions return an object of an unspecified type with some properties of a random access iterator:
  - Can be copy-constructed and copy-assigned
  - Can be compared for equality (`==` and `!=`)
  - Can be used in expressions `a + n, a - n, a - b`, where `a` and `b` are objects of the type and `n` is an integer value
- But most importantly, it can be passed to oneDPL algorithms

Simple Example of Using SYCL Buffer

Keeping data on the device with consecutive usage

```
#include <oneapi/dpl/execution>
#include <oneapi/dpl/algorithm>
#include <oneapi/dpl/iterator>
#include <sycl/sycl.hpp>
int main()
{
    sycl::buffer<int> buf{ 100000 };
    dpl::fill(dpl::execution::dpcpp_default, dpl::begin(buf),
              dpl::end(buf), 42);
    auto result = dpl::find(dpl::execution::dpcpp_default,
                            dpl::begin(buf), dpl::end(buf), 42);
    return 0;
}
```

Passing buffer access modes to the backend

```
auto buf_begin = dpl::begin(buf);
dpl::fill(my_policy, buf_begin, buf_begin + 1000, 42);
```

```
auto buf_begin = dpl::begin(buf, sycl::write_only, sycl::no_init);
dpl::fill(my_policy, buf_begin, buf_begin + 1000, 42);
```

How to Use a Zip Iterator

TriAdd example with oneDPL

Adding two vectors **B** and **C** multiplied by a scalar **s** to vector **A**:

A + = B + s * C

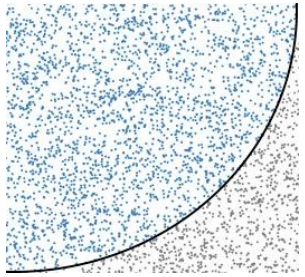
```
/* ... */
double *d_A = sycl::malloc_device<double>(length, q);
double *d_B = sycl::malloc_device<double>(length, q);
double *d_C = sycl::malloc_device<double>(length, q);
/* ... */
double scalar(3);
  {
    for (int iter = 0; iter <= iterations; iter++) {
      if (iter == 1)
        auto begin = dpl::make_zip_iterator(d_A, d_B, d_C);
        dpl::transform(dpl::execution::make_device_policy<class TriAdd>(q), begin,
                       begin + length, d_A, [=](const auto &t) {
                         auto [a, b, c] = t;
                         return a + b + scalar * c;
                       });
    }
  }
/* ... */
```

https://github.com/ParRes/Kernels/blob/default/Cxx11/nstream-onedpl.cc

UXL
Unified Acceleration Foundation

Monte Carlo Method

Estimating Pi with oneDPL Random Number Generation



```
int main() {
    // ...
    sycl::queue q{sycl::gpu_selector_v};
    auto my_policy = oneapi::dpl::execution::make_device_policy(q);
    auto sum = oneapi::dpl::transform_reduce( my_policy, oneapi::dpl::counting_iterator<int>(0),
                                oneapi::dpl::counting_iterator<int>(N), 0.0f, std::plus<float>{},
                                       [=](auto n){
                                           float local_sum = 0.0f;
                                           // Get random coords. Note: the sequence is skipped by 2*n*LOCAL_N elements
                                           oneapi::dpl::minstd_rand engine(SEED, 2*n*LOCAL_N);
                                           oneapi::dpl::uniform_real_distribution<float> distr(-1.0f, 1.0f);
                                           for(int i = 0; i < LOCAL_N; ++i) {
                                             float x = distr(engine), y = distr(engine);
                                             if (x*x + y*y <= 1.0f)
                                               local_sum += 1.0f;
                                           }
                                           return local_sum / (float)LOCAL_N;
                                       });
    float estimated_pi = 4.0f*(float)sum / N;
    // ...
}
```

UXL
Unified Acceleration Foundation

Key differences to C++20 range algorithms

Comparing to the C++20 range algorithms, **oneDPL parallel range algorithms** have the following key modifications:

a) The execution policy parameter

b) Take a range, not an iterator, for the output data

c) Require sized ranges with random access

Count words example with C++17 parallel algorithms

```
int word_count(const std::string& text) {

    // check for empty string
    if (text.empty()) return 0;

    // compute the number characters that start a new word
    int result = std::transform_reduce(std::execution::par,
        text.begin(), text.end() - 1,            // sequence of left characters
        text.begin() + 1,                        // sequence of right characters
        0,                                       // initial value for reduction
        [](int s1, int s2) { return s1 + s2; },  // reduce: sum values together
        [](char s1, char s2) {                   // transform:
            return int(!std::isalpha(s1)            // check if the right character starts a word
                       && std::isalpha(s2));
    });

    // if the first character is alphabetical, then it also begins a word
    if (std::isalpha(*text.begin())) ++result;

    return result;
}
```

Count words example with oneDPL parallel range algorithms

```
int word_count(const std::string& text) {

    // check for empty string
    if (text.empty()) return 0;

    // compute the number characters that start a new word
    int result = dpl::ranges::count_if(dpl::execution::par,
        std::views::zip(text | std::views::take(text.size() - 1), // sequence of left characters
                        text | std::views::drop(1)),              // sequence of right characters
        [](auto v) {                        // the predicate for counting:
            auto [s1, s2] = v;                // unpack zipped sequence elements
            return !std::isalpha(s1)          // check if the right character starts a word
                   && std::isalpha(s2);
        }
    );

    // if the first character is alphabetical, then it also begins a word
    if (std::isalpha(*text.begin())) ++result;

    return result;
}
```